DataShare — MVP

Soutenance projet 3

Chadili Ismaël

03/03/2027

Objectif du projet

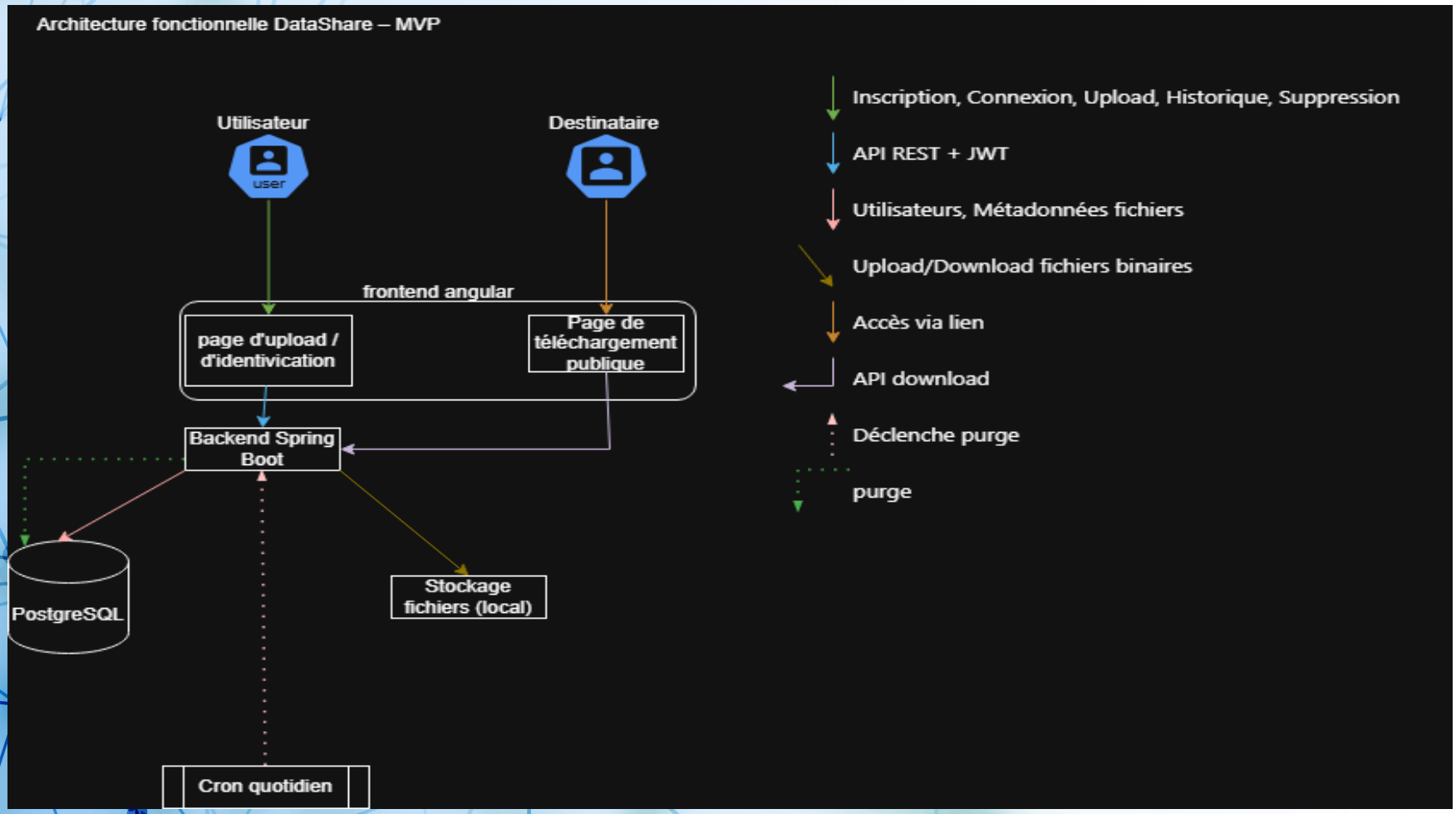
Application web de transfert de fichiers sécurisé

- MVP conforme aux US01 à US06 :
  - Upload avec compte
  - Téléchargement via lien public
  - Création de compte
  - Connexion utilisateur
  - Historique des fichiers
  - Suppression d’un fichier
- Livrer une solution robuste, testée, maintenable

Architecture globale

- Frontend : Angular (TypeScript)
- Backend : Spring Boot (Java 21)
- Base de données : PostgreSQL
- Stockage local des fichiers
- Authentification JWT
- Tâche planifiée (purge fichiers expirés)

Schéma architecture globale



Choix techniques

- Stack retenue :

  - Frontend : Angular + TypeScript

  - Backend : Spring Boot (Java 21)

  - Base de données : PostgreSQL

  - Stockage fichiers : local (filesystem)

  - Authentification : JWT

  - Tests : JUnit/Mockito (backend), Cypress (front, à compléter)

- Pourquoi ces choix ?

- - Rapidité de développement : outils et frameworks maîtrisés, documentation abondante
- - Robustesse : sécurité, validation, gestion des erreurs intégrées
- - Évolutivité : migration possible vers stockage cloud, ajout de fonctionnalités facilité
- - Facilité de maintenance : stack standard, communauté active, scripts de maintenance fournis

Explications des choix techniques

Élément	Technologie choisie	Alternatives possibles	Justification
Langage backend	Java 21	Node.js, Python, C#, PHP	Robuste, mature, excellent support Spring Boot, sécurité, maintenabilité, standard entreprise
Framework backend	Spring Boot	Express.js, Django, ASP.NET	Productivité, sécurité intégrée, gestion JWT, documentation API native, large communauté
Langage frontend	TypeScript	JavaScript	Typage fort, meilleure maintenabilité, standard pour Angular
Framework frontend	Angular 21	React, Vue.js	Structure modulaire, CLI puissant, intégration facile avec TypeScript, bonne gestion des forms
Base de données	PostgreSQL (SQL)	MySQL, MongoDB (NoSQL)	Fiabilité, requêtes avancées, transactions, open source, facile à héberger
Authentification	JWT (Spring Security)	OAuth2, Session, Firebase	Stateless, sécurisé, facile à intégrer avec Angular et Spring
Stockage fichiers	Local (filesystem)	S3, Azure Blob, GridFS	Simplicité MVP, pas de coût supplémentaire, migration cloud facile si besoin
Documentation API	OpenAPI (Swagger)	RAML, API Blueprint	Standard du secteur, supporté par Spring Boot et outils front
Tests backend	JUnit, Mockito	TestNG, Spock	Standard Java, documentation abondante, intégration Maven
Tests frontend	Vitest (unitaires), Cypress (e2e)	Jest, Jasmine, Playwright	Vitest rapide pour Angular, Cypress pour scénarios utilisateur complets
Gestion de dépendances	Maven (back), npm (front)	Gradle, Yarn	Standard du marché, intégration CI/CD, gestion efficace des dépendances
Outils dev	Git, GitHub, VS Code, Postman	GitLab, IntelliJ, Eclipse	Standards, intégration CI/CD, collaboration, tests API
Linters/Qualité	ESLint, Prettier, Checkstyle	SonarQube, TSLint	Automatisation du formatage et des bonnes pratiques, intégration facile dans CI
CI/CD	GitHub Actions	GitLab CI, Jenkins	Facile à configurer, gratuit pour projets publics, intégration directe avec GitHub
Suivi qualité/maintenance	TESTING.md, SECURITY.md, PERF.md, MAINTENANCE.md	README, Wiki, SonarQube	Documentation claire, traçabilité, conformité aux exigences du sujet
Utilisation IA	GitHub Copilot (pour doc + navbar)	ChatGPT, Copilot X, aucun	Respect du sujet : une US confiée à l'IA, supervision humaine, traçabilité dans la doc

Performance & Maintenance

- Budget de performance :
  - API (p95) < 500 ms (hors upload)
  - Upload petit fichier (p95) < 2 s
- Tests de charge (k6), suivi des temps de réponse et taux d'erreur
- Procédures de maintenance :
  - Mises à jour régulières des dépendances
  - Sauvegarde BDD et monitoring du stockage
  - Script de nettoyage des fichiers orphelins

Sécurité & Qualité

- Authentification JWT sur toutes les routes privées
- Mots de passe utilisateurs hashés (BCrypt)
- Contrôle d’accès propriétaire sur historique/suppression
- Validation serveur (taille, extension, expiration, mot de passe fichier)
- Gestion des erreurs : statuts HTTP explicites (401, 404, 410)
- Tests automatisés :
  - Unitaires et intégration backend (MockMvc)
  - e2e front
- Couverture de code backend : 75 %

Capture d'écran des tests

All files

71.24% Statements 394/553 **73.12%** Branches 166/227 **76.31%** Functions 58/76 **79.07%** Lines 291/368

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
app	100%	9/9	100%	10/10	100%	1/1	100%	3/3
app/core	98.24%	56/57	81.39%	35/43	100%	12/12	97.82%	45/46
app/pages	67.03%	303/452	67.28%	109/162	71.18%	42/59	75.41%	227/301
app/shared	74.28%	26/35	100%	12/12	75%	3/4	88.88%	16/18

datashare-backend Sessions

datashare-backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.datashare.backend.service		82 %		61 %	32	80	37	195	2	37	0	5
com.datashare.backend.security		31 %		10 %	9	13	25	44	4	8	0	2
com.datashare.backend.web.controller		56 %		0 %	4	13	12	33	3	12	0	3
com.datashare.backend.web		23 %		0 %	3	4	11	15	1	2	0	1
com.datashare.backend.domain.entity		83 %		n/a	6	36	10	56	6	36	0	2
com.datashare.backend		37 %		n/a	1	2	2	3	1	2	0	1
com.datashare.backend.config		100 %		n/a	0	10	0	22	0	10	0	1
com.datashare.backend.web.dto		100 %		n/a	0	8	0	8	0	8	0	8
Total	398 of 1 611	75 %	48 of 101	52 %	55	166	97	376	17	115	0	23

Created with JaCoCo 0.8.11.202310140853

```
PS C:\Users\the_m\Documents\formation_OC\projet3> npx playwright test frontend/tests/e2e/app.e2e.spec.ts

Running 5 tests using 1 worker

  ✓  1 frontend\tests\e2e\app.e2e.spec.ts:23:5 › Création de compte (1.6s)
  ✓  2 frontend\tests\e2e\app.e2e.spec.ts:34:5 › Connexion (558ms)
  ✓  3 frontend\tests\e2e\app.e2e.spec.ts:40:5 › Upload, vérification, téléchargement et suppression de fichier (5.6s)
  ✓  4 frontend\tests\e2e\app.e2e.spec.ts:100:5 › Navigation (746ms)
  ✓  5 frontend\tests\e2e\app.e2e.spec.ts:121:5 › Connexion avec mauvais mot de passe affiche une erreur (730ms)

  5 passed (11.3s)
```

```
TOTAL RESULTS

  checks_total.......: 300     9.746876/s
  checks_succeeded...: 100.00% 300 out of 300
  checks_failed......: 0.00%   0 out of 300

  ✓ status 201

  HTTP
  http_req_duration..............: avg=21.32ms min=10.99ms med=18.04ms max=62.82ms p(90)=32.1ms p(95)=46.79ms
    { expected_response:true }...: avg=21.32ms min=10.99ms med=18.04ms max=62.82ms p(90)=32.1ms p(95)=46.79ms
  http_req_failed................: 0.00%  0 out of 300
  http_reqs......................: 300    9.746876/s

  EXECUTION
  iteration_duration.............: avg=1.02s   min=1.01s   med=1.01s   max=1.06s   p(90)=1.03s  p(95)=1.05s
  iterations.....................: 300    9.746876/s
  vus............................: 10     min=10      max=10
  vus_max........................: 10     min=10      max=10

  NETWORK
  data_received..................: 190 kB 6.2 kB/s
  data_sent......................: 257 kB 8.3 kB/s
```

Difficultés & solutions

- Difficultés :

  - Sécurité : gestion fine des droits, validation JWT, hashage

  - Gestion fichiers : cohérence BDD/disque, suppression atomique

  - Tests : isolation des données, nettoyage entre tests, couverture des cas d’erreur

- Solutions :

  - Refactoring des services

  - Ajout de scripts SQL de maintenance

  - Utilisation de MockMvc et H2 pour les tests

Démonstration

- Parcours utilisateur :

  - Inscription
  - Connexion
  - Upload d’un fichier
  - Téléchargement via lien public
  - Suppression depuis l’historique

Conclusion & axes d’amélioration

- Points forts :

  - MVP complet, sécurisé, testé

  - Documentation et scripts de maintenance fournis

- Améliorations possibles :

  - Rate limiting sur login et download

  - Migration stockage S3

  - Monitoring avancé (logs, alertes)

Questions

- Merci pour votre attention et merci a mon mentor pour son accompagnement durant le projet
- Avez vous des questions ?